

Distributed Election Algorithms Survey

Mostafa Elshamy, Merna ElSaaran, Malak Zeerban, and Abdelrahman ElAskary

Computer Science and Engineering Department

The American University in Cairo, Cairo, Egypt

mostafaelshamy@aucegypt.edu, mernaelsaaran@aucegypt.edu,

malakzeerban@aucegypt.edu, elaskary03@aucegypt.edu

Index Terms—Distributed systems, Leader election, Consensus algorithms, Master-slave architecture, Self-stabilization, Gossip protocols, Fault tolerance, Herman's algorithm, Randomized algorithms, Distributed coordination.

I. INTRODUCTION

A Distributed System describes a type of multi-node computer network, in which a user requests a service by name [1]. In brief, this means that users of the system should be able to interact with the service provider in a transparent fashion (i.e., without knowledge of the underlying processes managing requests). In order to produce such a system, we must develop a method for Load Balancing, allowing for the service-providing nodes to distribute the workload amongst them.

There are a multitude of methods for load balancing, but this survey will focus on leader selection techniques. Leader selection refers to the process that occurs between servers to determine which amongst them will be responsible for fulfilling a user's request.

Our survey will first delve into the following leader selection techniques: Centralized Master-Slave Approach, Consensus-based Coordination, Gossip / Epidemic Protocols, and Distributed Election Algorithms.

After surveying and studying these methods, we will conduct a comparative analysis amongst them and determine a technique to use in our project. Furthermore, we will examine the various approaches to implementing our chosen technique and study the specific algorithms that could be used.

Our goal is to determine a specific algorithm for implementing a leader selection algorithm that we will implement in our project. Our selected method must be optimal for the use case that our project requires. Specifically, our system must sustain a small, peer-to-peer network that will primarily be used for image sharing. This means that individual requests may be quite large in size, considering the prevalence of high-quality image sharing and the subsequent encryption and decryption overhead.

II. BACKGROUND, Approaches to Leader Coordination

A. Centralized Master-Slave Approach

What it is: In a master-slave approach, also called leader-follower or primary-backup architecture, a single master node coordinates the system. It does that by serializing

writes, meaning that it collects all the write requests and lines them up in one order, and then assigning and scheduling work among the nodes, and finally it propagates state to the slave nodes, which execute tasks and/or serve reads.

This pattern is prevalent in replicated services, which are systems where the same data is stored on multiple machines and that could be for backup, fault tolerance, or load distribution purposes like databases with replicas, and cluster schedulers, which are tools that assign tasks to many servers in a computing cluster like Hadoop or Kubernetes controllers, because it provides a simple control locus and a single source of truth [2]–[4].

In production systems such as ZooKeeper, the distributed coordination service, the master broadcasts ordered state transitions to the slaves. If the primary fails, a new primary is established and recovery ensures a consistent prefix before resuming service [2]. Recent engineering case studies still adopt master-slave when it fits performance and operational constraints [3].

Advantages:

- **Simplicity & operational clarity:** All coordination flows through one node—the master—which makes the protocol's logic straightforward. In a master-slave system, the master decides the exact order in which all updates happen. This makes it easier to understand and build the system because every machine follows the same sequence of changes [2].
- **Consistency guaranteed by construction:** Routing all writes through the master yields a well-defined serialization point. Surveys of HTAP databases describe common deployments where a master node handles updates while secondaries replicate/serve reads, trading simplicity for scale on read-heavy workloads [5].
- **Performance:** If the master is designed such that it can send updates to all backups quickly, allowing the system to recover smoothly from failures, the master-slave setup can handle a large number of requests efficiently.

When it's a good fit:

- Small-to-medium clusters with moderate write rates, or when system designers favor operational simplicity over maximum scale.
- Services that already rely on a coordination/log replication layer like databases or analytics platforms with

one orchestrator and many workers—such as Hadoop or Spark [2], [6].

B. Consensus-based Coordination

What is it?: In distributed systems, consensus-based coordination refers to protocols that enable multiple nodes to agree on a shared value, decision, or system state despite failures, asynchrony in communication, or network partitioning. The goal is to achieve agreement, validity, and termination for all correct nodes so that the system behaves as a single entity uniformly [7]. This coordination model eliminates reliance on a single master node to enable distributed components to cooperate through a proposal–vote–commit cycle. The theoretical foundations of consensus date back to the Fischer–Lynch–Paterson (FLP) impossibility result [8], which proved that deterministic consensus cannot be guaranteed in a fully asynchronous system if even a single process may fail. Researchers developed timing assumptions and failure detectors to enable systems to advance in the face of partial uncertainty, making consensus practical. The earliest practical protocol, Paxos, proposed by Leslie Lamport, defined a rigorous method to reach agreement safely under all conditions and to guarantee liveness when timing conditions are favorable [9]. Although its complexity often makes appropriate implementation difficult, Paxos has since become a standard distributed consensus model. Raft was developed by researchers as a more straightforward, organized substitute for Paxos that would facilitate comprehension and application of consensus algorithms [10]. While one node assumes the role of a leader and manages all updates, the other nodes (followers) replicate the same ordered list of changes to maintain consistency. Raft and Paxos are now essential components of contemporary distributed systems. For instance, etcd, a crucial component of Kubernetes, uses Raft to maintain all nodes in a cluster aligned with the same configuration and state [11], and Google Spanner employs Paxos to maintain data synchronization across its global databases [12]. Consensus-based coordination has spread outside of cloud systems and conventional databases in recent years. New frameworks such as Delos introduced the idea of virtual consensus, allowing applications to switch between different consensus algorithms like Paxos and Raft without stopping system operations [7]. Meanwhile, the same principles are now used in blockchain and decentralized systems, where methods like Proof of Stake and Byzantine Fault Tolerance (BFT) allow participants to agree on shared ledgers without any central authority [13]. In short, consensus-based coordination has grown from a technique for reliability into the foundation of modern cloud, edge, and blockchain systems—ensuring that even when some nodes fail, all others can still agree and move forward together [7], [10].

C. Distributed Election

The final method we will be studying is Distributed Election Algorithms. A Distributed System is a collection of autonomous computing nodes which can communicate

with each other and which cooperate on a common goal or task [14]. A Distributed Election is a process that happens on those nodes, to balance the load of user requests. There exists a variety of methodologies on architecting a distributed election, and algorithms for conducting one, and our survey will examine this further on. The primary ethos behind distributed elections is that nodes should collaborate and compete for the determination of a leader, for the main purpose of providing fault tolerance. Peer-to-peer communication is essential, and such a topology allows for rapid communication, without the overhead of a message passing through a middleware or kernel. Some methods seek to synchronise nodes on a network, while others are asynchronous. Their objective is to guarantee that, despite node crashes or message delays, exactly one node functions as the leader at any given time (safety) and that the system eventually elects a leader when conditions permit (liveness). Classic algorithms provide different trade-offs between message complexity, time complexity, fault tolerance, and assumptions about synchrony. Well-known representative algorithms include the Bully algorithm (priority-based election), the Chang–Roberts ring algorithm (ring-topology extrema finding), and consensus algorithms like Paxos and Raft.

Mechanism: All forms of Distributed elections must maintain certain conditions, and follow a set procedure for conducting elections.

Each server/node must have a Unique Node Identifier, such as an integer ID or some sortable key. This allows nodes to break symmetry when multiple nodes are eligible to be leaders. There must be a way for nodes to detect when the leader (or any node needed for election) has failed (or at least appears to be unresponsive). Mechanisms like timeouts or heartbeats are common. This is particularly relevant on startup, as all nodes must be able to detect the absence of a leader (or the leader is suspected to have failed), and register themselves as candidates for leadership in the first election. Once an election has been initiated, candidate(s) send requests or messages to peers, which respond, granting vote or refusing depending on their internal state (e.g., whether they already voted, or whether they believe the candidate is up to date, or other metrics). Leader Selection must then occur, based on responses; a candidate becomes leader if it gets sufficient support. Often this means a majority or some quorum; in simpler election-only protocols it might be “highest ID reachable” or first in ring, etc. Other nodes transition to the follower state once the leader has been announced.

The leader periodically sends signals (heartbeats) or some keep-alive to all followers to show it’s still alive. Followers reset their failure detection timer on hearing heartbeats. If heartbeats stop for some period, followers assume the leader is down and start a new election. This incurs an overhead of message passing regularly across the network, so it can be coupled with other synchronisation activities to increase productivity.

A Distributed system must handle Concurrent Elections / Split Votes. Distributed elections must deal with the possibility that more than one node becomes a candidate at about the same time. A common architecture we could potentially use if we implemented this technique, would be to have as many processes running on each node, as there are nodes in the network. This would allow for the worst-case scenario that every node has called an election concurrently, and will also participate in every other election called.

The system must under reasonable assumptions guarantee that an election completes—i.e., a leader is elected within some bounded time.

The system must also Guarantee Uniqueness: at most one leader is elected for a given epoch or scope—no two nodes think they are leader for the same term/same group. Mechanisms like randomized delays, election terms, or conflict resolution rules avoid persistent livelock or two leaders (particularly in asynchronous networks).

Pros of Distributed Systems: One of the central advantages is the ability to automatically recover from leader failures by re-electing a new coordinator, avoiding single points of failure. Garcia-Molina’s foundational election paper discusses how, after a failure, nodes can reorganize by electing a new coordinator so the system continues useful work [14].

Similarly, the Amazon Builders’ Library says leader election “is a powerful tool for improving efficiency, reducing coordination, simplifying architectures,” but also warns that failure detection must be robust [15].

Election algorithms avoid having a fixed, hardcoded master; instead, leadership emerges dynamically based on availability and runtime conditions. This decentralization increases resilience in changing environments. The survey “A Survey and Taxonomy of Leader Election Algorithms in Distributed Systems” emphasizes that election algorithms must adapt to node failures and dynamic topologies [16].

Symmetric Design & Flexibility: Because all nodes run the same algorithm (symmetry), the system is more flexible: new nodes can join and be considered as eligible leaders. Election protocols often assume this symmetry in their formulation. The survey by Soleimanian and Arjang classifies algorithms under varying assumptions (e.g., about whether nodes know network size, topology) and shows flexibility in handling dynamic networks [16].

When implemented in a controlled environment (low latency, reliable network, small cluster), election algorithms can be simpler and more lightweight than full consensus layers. Some improved election algorithms reduce message complexity or shorten election times [17].

Disadvantages: Message Overhead / Communication Complexity. Many election protocols require multiple rounds or broadcast-style messaging, which scales poorly with increasing node counts. In worst-case scenarios, Bully can incur $O(n^2)$ messages when many nodes initiate elections [14]. Ring-based algorithms also may send messages proportional to network size; in the Chang–Roberts ring protocol, worst-

case message count is $3n - 1$ (for n nodes) under typical assumptions [18].

Election Latency / Downtime During Transitions. During a leader election, no node may have authoritative control, which leads to temporary unavailability or latency spikes. The system must wait until the election completes. Moreover, if elections are frequent (due to unstable networks), this overhead compounds.

Split-brain and Partition Problems. In the presence of network partitions or message loss, two disjoint sets of nodes might each elect their own leader (a split-brain), violating the “single leader” safety guarantee. Resolving such conflicts is complex. Amazon’s discussion mentions “new failure modes” as a drawback of leader election [15].

In fully asynchronous systems (no bounds on message delay or process speeds), it is impossible to guarantee both safety and liveness of leader election without additional assumptions (e.g. reliable failure detectors or randomization). This is analogous to the well-known FLP impossibility in consensus [8]. Some modern election protocols (e.g. PALE) try to relax assumptions, but they come with tradeoffs [?].

D. Gossip / Epidemic Protocols

Gossip, or epidemic, protocols are a family of probabilistic, decentralized algorithms inspired by the spread of biological viruses or rumors among human populations. In distributed systems, they allow information to be disseminated quickly and reliably without a central leader or coordinator. Each node in the system exchanges state (e.g., updates, membership, or configuration data) periodically with randomly selected peers. This random and repeated peer communication over time ensures with high probability that information from any node is disseminated to all nodes in the system [19].

The operation of gossip protocols usually comprises the following steps: *Periodic Rounds:* Each node randomly selects one or more peers in every round (a fixed time interval) and exchanges information with them. *State Update/Exchange:* Nodes accept updates, merge them with their local state, and possibly propagate fresh information to others in subsequent rounds. *Probabilistic Convergence:* Through repeated, redundant dissemination, information “infects” the network somewhat like an epidemic, ensuring eventual dissemination even in the face of node or network failures [20].

Variants of gossip protocols include anti-entropy (pairwise reconciliation of state), rumor mongering (probabilistic propagation of a given message), and push-pull models (nodes both push and pull updates in an exchange). Modern distributed systems employ gossip for failure detection, system configuration, distributed hash tables, partition healing, and large-scale group management [21].

Advantages: Scalability Gossip protocols possess communication and computation patterns that scale well, at least logarithmically, in the number of nodes. This renders them an ideal fit for large peer-to-peer systems, data stores at a global scale, and sensor networks [1]. *Fault Tolerance and Robustness:* Decentralized and redundant, they have high

resilience to both node failure and transient network partitions. The protocol will continue to work as long as the network is at least partially connected and a sufficient percentage of nodes are participating [1].

Self-Organization and Adaptivity Gossip protocols achieve simple membership change—nodes can join or leave the group as they please. The system self-organizes, with new or re-joining nodes automatically re-integrated without external coordination [1]. *Simplicity and Uniformity*: It is simple to implement: each node independently runs the same logic, randomly selecting peers and aggregating state. Lack of need to assign, monitor, or switch leadership simplifies code as well as protocol correctness arguments [1]. *Eventual Consistency*: Gossip protocols natively support eventual consistency—the guarantee that, in the absence of new updates or continual failures, all correct nodes will eventually converge to the same state. This weak consistency model is typically sufficient for most large-scale applications for which absolute immediacy is less critical [1]. *Application Diversity*: These protocols have widespread applications in numerous systems, such as Amazon Dynamo (eventual consistency for distributed key-value storage), Cassandra, peer-to-peer overlays, distributed monitoring (failure detectors), and configuration management in datacenters today [22].

Limitations: Weak Consistency Guarantees: Data is disseminated in a best-effort, probabilistic manner. Nodes may momentarily disagree or have stale state. Strong consistency, viz., linearizability or sequential consistency, is not attainable by gossip alone; other coordination mechanisms must be employed for such properties [1]. *Convergence Delay and Predictability*: Convergence time for the whole network differs according to network size, connectivity, and gossip round frequency. While typically logarithmic in theory, practical convergence can be highly unstable, especially with node churn or biased peer selection [20]. *Message Overhead and Network Traffic*: Despite efficient scaling, repeated message exchanges result in ongoing bandwidth and processing overhead. By definition, a large amount of data is duplicated and retransmitted, potentially taxing network resources in constrained environments [1]. *Risk of Message Loss and Staleness*: During network failures or slow links, parts of the network may receive updates late or never. State divergence or “network split-brain” may occur if nodes are partitioned for a long period of time [1]. *Limited Applicability to Coordination-Intensive Tasks*: Problems that demand strict sequencing, strong agreement, or ordered execution across nodes (e.g., mutual exclusion, state machine replication, or atomic commitment) are not solvable through gossip alone and necessitate consensus protocols (e.g., Paxos or Raft) [10]. *Security*: Without cryptographic protection, gossip protocols are theoretically vulnerable to malicious adversaries capable of overwhelming the network with spurious updates or fabricated states, attacking integrity and availability.

Example Scenario: Take a distributed key-value store running on hundreds of virtual or physical machines. When there is a configuration change (e.g., adding a new node or modi-

fying a replication factor), the change can be advertised in a gossip message. Each node, when it learns the change, gossips it to a few randomly selected peers. Within a few rounds, all reachable nodes learn and adopt the change, enabling graceful and scalable reconfiguration. When a node crashes or is unplugged, its failure is observed by peers and gossiped throughout the network, triggering the repairs or removals needed without centralized reconfiguration or failure detection services.

Summary: Gossip/epidemic protocols are an essential means of scalable, robust, and adaptive coordination for large and dynamic distributed systems. While their “leaderless” and probabilistic nature adapts them suitably to efficient information dissemination and membership management, they are not a panacea. Practitioners will have to weigh their probabilistic guarantees and message overhead carefully against application requirements, especially where strong consistency or close coordination is necessary. These features explain why gossip protocols are today essential parts of modern distributed applications, especially where scalability, robustness, and self-organization are crucial.

Justification of using Distributed Elections

Distributed election algorithms provide a middle ground between very simple centralized coordination and full-blown consensus or gossip-based systems. Their mechanism ensures that one node acts as the leader, which gives you decisive control for load distribution and task coordination, but they are lighter weight than full consensus protocols in terms of message overhead and implementation complexity.

Given our project: we have multiple server nodes, a client/middleware, and we need exactly one node to be chosen as the leader to acknowledge work and avoid duplicate processing. Other requirements include moderate network reliability (LAN or connected PCs), low to moderate scale (just a few nodes), and the need for predictable (reasonable) latency in leader election.

Distributed election algorithms fit this scenario well because:

- They provide unique, authoritative leadership without requiring full consensus on every operation. This reduces complexity but still meets the need for coordination and avoiding duplicate work.
- They have lower message and implementation overhead than consensus protocols, which means you can write or integrate them more quickly and keep things responsive.
- They tolerate leader failures (since leadership is dynamic), which gives you resilience that centralized systems lack.
- They are simpler than gossip-based protocols in enforcing consistency of decisions, since gossip doesn’t give you a guarantee of exactly one leader at a time (gossip is better for spreading status, not coordinating who does what first).

III. DISTRIBUTED ELECTION TECHNIQUES

A. Bully Algorithm

Mechanism: The Bully Algorithm is a classical leader election algorithm for synchronous distributed systems. It is used to select a coordinator (leader) from among a set of distributed processes, with each process having a unique identifier (ID). The process with the highest ID is ultimately selected as the coordinator. The algorithm pre-assumes that processes are able to communicate reliably, and failures can be detected through timeouts.

The Bully Algorithm operates as follows:

Election Initiation: When a process discovers the current coordinator to have failed, typically by a timeout, it initiates an election by sending an "election" message to all processes with higher IDs [1]. **Response Phase:** The processes with higher IDs, if alive, respond with an "answer" message, confirming their presence, and thus initiate their own elections by sending election messages to even higher-ID processes [1]. **Coordinator Election:** An election process that is initiated, if it doesn't receive any answer messages within some timeout, presumes that it has the highest ID of active processes and declares itself the coordinator [1]. **Coordinator Announcement:** The new coordinator multicasts a "coordinator" message to all other processes to declare its leadership [1]. **State Update:** Upon receiving the coordinator message, processes update records to recognize the new leader and normal operation resumes.

This cascading series of elections and response messages ensures that eventually the process with the highest operational ID becomes the leader. The algorithm terminates when all processes have responded to the newly elected coordinator [1].

Pros: **Deterministic Leader Election:** The algorithm guarantees that the process with the highest ID is elected as the coordinator, so the election result is deterministic [1]. **Fault Detection using Timeouts:** By utilizing timeout mechanisms, the algorithm effectively detects the failure of the coordinator and initiates recovery by triggering new elections [1]. **Simple and Intuitive:** Theoretical functioning of the Bully Algorithm is easy, making its implementation and comprehension relatively easy [1]. **Handles Multiple Concurrent Elections:** More than one process may detect coordinator failure and continue with elections simultaneously without causing system inconsistency [1]. **The process having the highest ID will ultimately win.** **Appropriate for Small to Medium-Sized Systems:** For systems with a modest number of processes, message overhead and election time are reasonable.

Cons: **High Message Complexity:** In the worst case—when the process with the lowest ID starts an election—the Bully Algorithm can generate messages, where is the number of processes. This can result in high network overhead for large systems [1]. **Dependence on Synchronous Assumptions:** The algorithm relies on accurate timeout values and assumptions on message transmission delays. If these assumptions are invalidated (e.g., due to network partitions or extremely long delays), erroneous failure detections or multiple simultaneous coordinators can occur [1]. **Single Point of Bottleneck:** The

highest-ID process "bullies" its way into leadership by overriding others. This can lead to temporary overloading of higher-ID processes with election messages [1]. **Recovery from Failure Replacements Problem:** If a crashed process is restarted with the same ID by another process, concurrent elections may lead to split brain scenarios in which two or more coordinators may exist briefly [1]. **Poor Scalability:** Quadratic message complexity and timeout dependence make the Bully Algorithm unsuitable for very large or highly dynamic distributed systems [1].

B. Ring Election Algorithm

Mechanism: The Ring Election Algorithm is the canonical pattern known for distributed election. Processes form a logical ring and each of the processes knows its successor. Upon suspecting the leader has failed, any node can start an election by sending a message with its ID to its successor. Each node forwards and may update the "best-so-far" ID under the algorithm's rules. An example of evaluating the best ID is for it to have the highest value, for example. When the originating node receives back its own ID, which must be the maximum since it received it again, it declares victory and circulates a coordinator victory message so every participating node updates the information of the leader to that process/node accordingly.

Numerous IEEE works analyze and improve this scheme with its initial appearance (see table I as the Chang–Roberts algorithm, then evolving to include the classic linear-message ring election and fault-resilient election on dynamic rings [23], [24]. The development of ring-based leader election algorithms shows how systems evolved from simple but inefficient designs to fast and fault-tolerant ones. The linear-message ring algorithm introduced at IEEE HICSS 1991 reduced the cost from $O(n^2)$ in the original Chang–Roberts method to just $O(n)$ by letting each node forward only one election message around the ring [24]. Later, Chan and Chin's IEEE TPDS 1993 algorithm improved resilience by handling node crashes and recoveries, proving that elections could complete in $\Theta(n \log n + k_r)$ messages, where k_r is the number of recovered nodes [23]. The IEEE FTCS 1994 chordal-ring version added shortcut links between non-adjacent nodes, further reducing delay and tolerating multiple failures with a worst-case cost of $O(n \log n)$ [23]. Finally, modern self-stabilizing algorithms such as PraSLE (IEEE EDCC 2021) extend these ideas to dynamic IoT networks, allowing rings to automatically recover from any temporary faults and still agree on a single leader [25].

Pros:

- **Decentralized and lightweight:** Only local neighbor communication is required, making it useful in constrained topologies (e.g., sensor rings or bus-like networks) [24].
- **Deterministic winner under IDs:** The highest-ID node emerges cleanly, avoiding tie-breaking ambiguity [23], [24].

TABLE I
EVOLUTION OF RING-BASED LEADER ELECTION ALGORITHMS

Era	Focus	Key Improvement	Complexity
1979 (Chang–Roberts)	Simple correctness	Works, but inefficient	$O(n^2)$
1991 (HICSS)	Linear-Message Ring Algorithm	Efficiency: Reduced to one pass	$O(n)$
1993 (TPDS)	Fault-Recovery Algorithm	Fault recovery: Handles crashes	$\Theta(n \log n + k_r)$
1994 (FTCS)	Chordal Ring Election	Scalability: Added shortcuts	$O(n \log n)$
2021 (EDCC)	PraSLE	Self-healing: Auto-recovery, IoT-ready	Depends on topology, convergent

Cons:

- **Topology assumptions:** A stable logical ring is required; mobility or partition dynamics complicate correctness and liveness.
- **Latency under scale:** Even optimized variants traverse the ring — $O(n)$ time is intrinsic for propagation; worst-case message overhead can be high in less-optimized schemes.
- **Fragile under failures unless extended:** Broken links or crashed nodes stall the election unless the protocol incorporates failure detection and skip/reconnect logic; dynamic-ring analyses quantify the extra cost [23].

Modern Twists & Where Rings Still Appear: Recent IEEE-published work explores leader election for constrained or dynamic environments rather than large-scale datacenters.

- **Self-stabilizing IoT leader election (EDCC 2021):** PraSLE targets resource-constrained devices and converges from arbitrary states; useful where devices can reboot or communication is lossy [25].
- **Dynamic networks (NCA 2021):** Centrality-based eventual leader election drops ring assumptions and elects a leader using graph-centrality signals, better matching dynamic, opportunistic topologies [26].

C. Paxos Consensus Algorithm in Distributed Systems

Introduction and Definition: Consensus in distributed systems means that a set of processors agree on a single value regardless of failures. By distributing a state machine among servers, Lamport’s Paxos method resolves consensus issues for asynchronous networks that experience crash failures. Paxos ensures that all non-faulty replicas agree on the sequence of client commands even when some servers crash. IEEE papers note that Paxos is a state-machine replication protocol used in production systems such as Google’s Chubby lock service, Microsoft’s Autopilot, and Ceph [9], [27].

Paxos belongs to the family of Multi-Paxos (multi-decree Synod) algorithms that repeatedly run single-decree consensus to build a consistent log of commands [28]. Variants like Cheap Paxos and the Paxos Register further optimize the original algorithm while preserving its fundamental properties [29].

Mechanism of Paxos: Paxos divides the processors into three logical roles — proposers, acceptors, and learners — and uses a leader to coordinate them [9].

Agents and Safety Conditions: Proposers suggest values, acceptors vote to choose a value, and learners learn the outcome. Paxos achieves:

- **Non-triviality:** Only suggested values can be chosen.
- **Consistency:** At most one value is chosen.
- **Conservatism:** Chosen values are eventually learned [9].

Quorums and Ballot Numbers: Each leader has a unique set of ballot numbers and uses them in coordinating instances of agreement. Acceptors are formed into intersecting quorums (majorities); intersection ensures that different proposers cannot resolve conflicting values [9].

The traditional Paxos algorithm operates in two phases:

- 1) **Phase 1 (Prepare/Promise):** The leader selects a new distinctive ballot number and sends a *Prepare* request to acceptors. Each acceptor replies with a *Promise* not to accept proposals with lower numbers and sends back any previously accepted value.
- 2) **Phase 2 (Propose/Accept):** After obtaining promises from a quorum, the leader sends an *Accept* request with a proposed value. Acceptors respond with *Accept* messages if they have not promised a higher ballot number. Once a quorum accepts the same value, consensus is reached, and learners record the decision [9], [28].

Multi-Paxos and Paxos Register Abstraction: In practice, multiple decisions are required. Multi-Paxos designates a long-lived leader to bypass Phase 1 for most decisions, reducing message delays. The Paxos Register abstraction presents a write-once register model where processes perform reads and writes guarded by tokens. A leader must obtain a token by reading before it can write, ensuring that new leaders only propose values consistent with prior ones [29].

Advantages of Paxos:

- **Fault Tolerance and High Availability:** Paxos provides a reliable method to maintain consistency despite crash failures. Using quorums, it remains correct even if some acceptors are faulty; a leader can make progress when it can communicate with a majority [9], [28]. Paxos is used in highly available systems such as Chubby, Autopilot, and Ceph [27], [30].
- **Safety under Asynchronous Networks:** The algorithm guarantees safety—non-triviality, consistency, and conservatism—regardless of message delays or reordering. Safety holds even when multiple leaders exist; progress only requires eventual leader stability [9], [28].
- **Modularity and Formal Reasoning:** Paxos can be presented as a sequence of consensus instances, enabling modular reasoning about complex distributed systems. The Paxos Register abstraction simplifies reasoning by

hiding asynchronous communication and quorum complexity [29].

- **Extensibility to Variants:** Paxos serves as the foundation for numerous variants. Cheap Paxos reduces the number of full-function servers required for fault tolerance by introducing auxiliary processors [31]. Multi-Paxos, Byzantine Paxos, and Disk Paxos extend the core algorithm while retaining safety guarantees [28], [31].
- **Industry Adoption:** The widespread use of Paxos in large-scale systems—such as Google, Microsoft, and Ceph—demonstrates its robustness and practical efficiency [27], [30].

Limitations of Paxos:

- **Implementation Complexity:** The algorithm’s correctness proofs are subtle, and its implementation is challenging. Studies note that the complete Paxos protocol is non-trivial for practitioners [31].
- **Leader Election and Progress:** Paxos’s liveness depends on electing a unique and stable leader to make progress. Under high asynchrony or frequent leader changes, progress can be delayed or halted [9], [31].
- **Performance Overheads:** Paxos requires two message rounds (prepare/promise and propose/accept) and durable logging, resulting in higher latency compared to View-stamped Replication (VSR) and Zab protocols, which apply speculative updates [32], [33].
- **Synchronous Clock Requirement for Read-Only Optimizations:** Some implementations use leases for read-only requests, requiring clock synchronization to prevent stale reads [28].
- **Memory and Garbage Collection:** During recovery, Paxos may need to replay many commands or transfer full state. Without garbage collection, stored logs can grow significantly [34].
- **Not Speculative:** Unlike VSR and Zab, Paxos does not apply speculative execution. This simplifies correctness but delays responses, as every update must be fully decided before execution [32].

D. Raft Algorithm

Raft Algorithm Mostafa

Invention: In 2014, a group of researchers at Stanford University decided that implementing Paxos was far too complex, and very few people truly understood all aspects of it. They set out with the express intention of developing a distributed election algorithm that was simple to understand and replicate, whilst maintaining the reliability, consistency and results of Paxos. In 2014 they released “In Search of an Understandable Consensus Algorithm”, Authored by Diego Ongaro and John Ousterhout [10], exhibiting the Raft algorithm, a new leadership election and log replication algorithm that was not complex, digestible for students, with similar performance as Paxos. Their seminal paper on Raft won them Best Paper at the 2014 USENIX conference, and Raft has become one of the most widely used systems for distributed elections. It is the backbone of Kubernetes, employed by major industry titans

like Google and Amazon Web Services (AWS) [15]. Since its invention, there have been many updates and improvements to the Raft algorithm, which we will get into in this section.

Leader Election: The mechanism for electing a leader is as follows. Each node in the network begins as a state follower, with a countdown timer. Followers anticipate a heartbeat signal from their leader node before they time out, else they change states to candidates. Once a node becomes a candidate for leader, they send out a request for votes signal from all other nodes. If a candidate receives at least the majority of votes (of the total number of nodes on the network), it becomes leader [35]. The leader must then send out a heartbeat signal to all nodes, indicating that it is the leader, and initiating log replication.

Log Replication: Log replication is the primary motivation for leader elections, as maintaining a consistent log of events on the system, and updating it across the network is crucial for any simple or robust distributed system. The leader appends client commands to its own log (or entries) and information about the previous log index and term, and replicates these entries to follower nodes using AppendEntries RPCs. Once an entry is safely stored by a majority, it is considered committed and applied to the state machines on all nodes. Followers check if their own logs match the leader’s up to the previous log index and term. If not, they reject the entry, prompting the leader to retry with earlier log entries until the follower’s log matches (this is called log backtracking). The leader tracks how many followers have replicated each entry (through acknowledgments). When a log entry has been replicated on a majority of nodes (including the leader), it is considered committed.

Handling Failures and Recovery: If a follower crashes or falls behind, when it rejoins, the leader will send it all missing log entries (using the same AppendEntries mechanism) until it catches up and is consistent with the leader. This catch-up process ensures that even after failures, all non-faulty nodes eventually converge to the same log and state. The inherent mechanisms of the log replication, and validation of the leader election, allow for continuity even in the case of multiple failed servers, split elections, and inconsistent logs. As such it has been used in a wide variety of systems and applications. Raft’s clear separation of leader election and log replication, alongside use of randomized timeouts and easy-to-understand transitions (leader, follower, candidate), [10] makes the core protocol tractable for practitioners and researchers.

Iterations and developments: Multi-Strategy Leader Election [36]: Enhancements allow followers to proactively call new elections if current leaders become slow, reducing downtime and improving throughput without increasing election time. Weighted and Federated Raft: Modifications to accommodate heterogeneous nodes, such as those in blockchain or IoT use cases, by assigning weight to nodes or leveraging federated learning for performance improvement. Leadership Transfer and Lease-Based Election: To avoid split votes or unnecessary elections, variants introduce explicit leader transfer or time-based leadership leases for smoother failover and

reduced churn. Performance Optimizations: Many real-world deployments employ batching, pipelining, and RPC compression techniques to improve throughput and reduce latency. Resilience Workarounds: Some systems add extra logic to prevent “livelocks” during network partitions or make log divergence less likely in adverse conditions.

E. Herman’s Algorithm

Mechanism: Herman’s algorithm is a self-stabilizing randomized token-passing algorithm originally designed for ring topologies. Each process in the ring may hold a token, and the system evolves in synchronous rounds. In every round, a process that has a token either keeps it or passes it to its clockwise neighbor with equal probability (typically $\frac{1}{2}$). When two tokens meet at a single node, they annihilate each other, gradually reducing the number of tokens in circulation until only one token remains—the leader. This mechanism guarantees that, regardless of the initial configuration, even if the network starts with multiple tokens or inconsistent states, the system will eventually converge to a legitimate state with exactly one token. That property defines its self-stabilization behavior [37].

IEEE works such as Flatebø and Datta (IEEE TSE, 1994) first demonstrated the feasibility of self-stabilizing token ring algorithms under deterministic and probabilistic behaviors [38]. These designs influenced Herman’s approach by formalizing token circulation, collision, and recovery mechanisms in constrained topologies. Later IEEE studies like Derhab et al. (IEEE TDSC, 2008) extended these foundations to fault-prone networks, showing that leader election can remain self-stabilizing even under frequent transient faults by allowing nodes to detect inconsistencies and rejoin without restarting the system [39].

The most detailed theoretical analysis, Kiefer et al. (2011), established that the expected stabilization time of Herman’s algorithm is bounded by $0.64n^2$, where n is the number of nodes, and also presented an asynchronous version achieving convergence under relaxed timing assumptions [37]. These results confirmed Herman’s algorithm as one of the simplest yet most elegant examples of probabilistic self-stabilization in distributed systems.

Advantages:

- **Self-stabilizing:** The system can fix itself and return to a correct state without needing a restart or central control.
- **Simple topology:** Works efficiently on ring networks using only neighbor-to-neighbor communication.
- **Fair and deadlock-free:** Random token passing avoids conflicts and ensures every node has a fair chance.
- **Lightweight:** Each node only needs to store a small amount of information (one token bit).

Limitations:

- **Slow recovery time:** It takes roughly $\Theta(n^2)$ steps to stabilize, which becomes inefficient as the network grows.
- **Unpredictable timing:** Because of randomness, the exact time to reach one leader varies each run and is non-deterministic.

- **Limited fault handling:** Cannot handle permanent node failures; primarily tolerates transient faults.

Modern Extensions and IEEE Relevance: Recent IEEE studies have extended self-stabilizing and randomized leader election beyond simple ring networks to handle dynamic and fault-prone systems, see table for more historic information II:

- **IEEE TDSC 2008 (Derhab et al.) — Fault-Prone Networks:** Proposed a self-stabilizing leader election method that can recover from frequent temporary faults without restarting the whole network [39].
- **IEEE EDCC 2021 (Conard & Ebnenasir, PraSLE) — IoT Environments:** Applied self-stabilizing principles to IoT and low-power devices, ensuring the system can still reach a stable leader even when links fail or devices restart [25].
- **IEEE Access 2020 (Kandhouli et al.) — Dynamic Graphs:** Combined randomization and consensus techniques to enable leader election in changing network topologies where connections constantly appear or disappear [40].

F. Summary of Distributed Election Algorithms

Observe Table III

IV. CHOICE FOR OUR PROJECT

Our Choice Raft’s suitability for our project’s fault tolerance testing stems from its ability to reliably and deterministically select a leader, which is essential when orchestrating controlled node failures. In our peer-to-peer network, Raft’s consensus-driven approach ensures that, at any given moment, the system can unambiguously identify the leader node. This property is crucial for simulating failures and observing system response to coordinator absence. The protocol’s design guarantees that the remaining nodes will promptly detect the leader’s failure and initiate a new election, thereby maintaining system availability and correctness throughout the experiment.

The inherent security in maintaining strong consistency and availability during and after failures means that our simulations will closely mirror the behavior of real-world distributed systems, providing results that are both robust and relevant. In contrast, simpler algorithms such as Bully are more susceptible to issues like multiple leaders or inconsistent state during network partitions, which can undermine the reliability of our testing.

By employing Raft, our project benefits from a protocol that is not only widely adopted in industry but also specifically engineered to handle the types of failure and recovery scenarios we aim to simulate, ensuring our fault tolerance tests are both realistic and representative of modern distributed environments. It would be nonsensical to consider employing Paxos at this stage, and a ring architecture would not be as fault tolerant in our network and use case, considering the requirements of our project and the class demonstration. Hence, the majority of their iterations that were surveyed and analysed above may be disregarded in favor of using Raft throughout our system. This eventually leads to less overhead,

TABLE II
EVOLUTION OF HERMAN’S ALGORITHM AND SELF-STABILIZING LEADER ELECTION

Era	Focus	Key Improvement
Early (1990)	Randomized token circulation	Introduced probabilistic token annihilation ensuring self-stabilization
IEEE TSE (1994)	Simplified token-ring stabilization	Deterministic and probabilistic two-state models enabling convergence
IEEE TDSC (2008)	Fault-prone network resilience	Extended stabilization to tolerate transient faults and dynamic joins
Modern Analysis (2011)	Quantitative convergence proofs	Bounded expected stabilization time to $0.64n^2$, introduced asynchronous variant

TABLE III

Feature	Bully	Ring	Paxos	Raft	Herman
Message Complexity	$O(n)$ per phase	$O(n)$	$O(n^2)$	$O(n)$ per election phase	$O(n \log n)$
Scalability	Moderate	Good for moderate n	Moderate	Moderate	Moderate
Ease of Implementation	Complex	Simple	Complex	Moderate	Simple / randomized
Synchronization Model	Synchronous	Synchronous	Asynchronous	Asynchronous	Asynchronous
Communication Topology	Fully connected	Logical ring	Fully connected P2P	Fully connected	Ring & token-passing

higher efficiency, and simpler implementation for the team. Limitations

Raft’s limitations are as much a reflection of its design philosophy as they are technical constraints. The single-leader model, while elegantly simple, means every write and log replication must pass through one node, making the leader a potential bottleneck as clusters scale and workloads intensify. Further, queueing model analyses reveal that Raft’s performance under heavy load is sensitive to the number of follower nodes and their service capacity. Increasing the number of followers can reduce latency and message delay, but Raft’s single-leader bottleneck remains a limiting factor for ultra-high concurrency scenarios. In contrast, newer Raft variants—such as TD-Raft—have demonstrated significant improvements, reducing the average number of elections by over 70% compared to traditional Raft, which helps stabilize throughput and minimize downtime during spikes.

As such, we will be sure to implement the most recent iteration of the Rust algorithm in our design, to gain the full benefit of this choice.

REFERENCES

- [1] G. F. Coulouris, J. Dollimore, T. Kindberg, G. Blair, A. K. Bhattacharjee, and S. Mukherjee, *Distributed Systems: Concepts and Design*, 5th ed. Boston, MA: Addison-Wesley (Pearson), 2012.
- [2] F. P. Junqueira, B. C. Reed, and M. Serafini, "Zab: High-performance broadcast for primary-backup systems," in *IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2011, pp. 245–256.
- [3] X. Chen and W. Zhang, "Engineering case study on primary-backup architectures in cloud coordination services," *IEEE Computer*, vol. 57, no. 4, pp. 50–61, 2024.
- [4] Y. Li, F. Zhao, and H. Liu, "Cluster scaling and leader coordination in distributed systems," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 11, pp. 4120–4132, 2024.
- [5] W. Zhang, Y. Liu, J. Chen, and L. Xu, "A survey of hybrid transactional/analytical processing (htap) databases: Architecture, consistency, and performance," *ACM Computing Surveys*, vol. 56, no. 3, pp. 1–35, 2024. [Online]. Available: <https://doi.org/10.1145/3624217>
- [6] C. Zhang, G. Li, J. Zhang, X. Zhang, and J. Feng, "Htap databases: A survey," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 11, pp. 6410–6429, 2024.
- [7] M. Balakrishnan, D. Malkhi, J. H. Howard, and T. Wobber, "Delos: Virtual consensus for consensus-as-a-service," in *USENIX OSDI*, 2020.
- [8] M. J. Fischer, N. A. Lynch, and M. S. Paterson, "Impossibility of distributed consensus with one faulty process," *Journal of the ACM*, vol. 32, no. 2, pp. 374–382, 1985.
- [9] L. Lamport, "The part-time parliament," *ACM Transactions on Computer Systems*, vol. 16, no. 2, pp. 133–169, 1998.
- [10] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in *2014 USENIX Annual Technical Conference (USENIX ATC 14)*, Philadelphia, PA, 2014, pp. 305–319. [Online]. Available: <https://www.usenix.org/conference/atc14/technical-sessions/presentation/ongaro>
- [11] etcd Authors, "etcd documentation," <https://etcd.io/docs/>, 2017.
- [12] J. C. Corbett, J. Dean, M. Epstein, A. Fikes, C. Frost, J. J. Furman, S. Ghemawat, A. Gubarev, C. Heiser, P. Hochschild, W. Hsieh, S. Kanthak, E. Kogan, H. Li, A. Lloyd, S. Melnik, D. Mwaura, D. Nagle, S. Quinlan, R. Rao, L. Rolig, Y. Saito, M. Szymaniak, C. Taylor, R. Wang, and D. Woodford, "Spanner: Google's globally-distributed database," in *Proceedings of the 10th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2012.
- [13] B. Cao, Y. Li, H. Zhao, J. Zhang, and S. Wan, "Toward secure and efficient consensus mechanisms for blockchain-based systems," *IEEE Network*, vol. 33, no. 4, pp. 38–45, 2019.
- [14] H. Garcia-Molina, "Elections in a distributed computing system," *IEEE Transactions on Computers*, vol. C-31, no. 1, pp. 48–59, 1982.
- [15] Amazon Web Services, "Leader election in distributed systems," <https://aws.amazon.com/builders-library/leader-election-in-distributed-systems/>, 2020, accessed: 2025-10-09.
- [16] F. S. Gharehchopogh and H. Arjang, "A survey and taxonomy of leader election algorithms in distributed systems," *Indian Journal of Science and Technology*, vol. 7, no. 6, pp. 815–830, 2014.
- [17] P. B. Soundarabai, J. Thriveni, K. R. Venugopal, and L. M. Patnaik, "An improved leader election algorithm for distributed systems (merea)," *International Journal of Next-Generation Networks (IJNGN)*, vol. 5, no. 1, pp. 21–30, 2013.
- [18] W.-M. Chen, "Cost distribution of the chang-roberts leader election algorithm and related problems," *Theoretical Computer Science*, vol. 369, no. 1–3, pp. 442–447, 2006.
- [19] A. Demers, D. Greene, C. Hauser, W. Irish, J. Larson, S. Shenker, H. Sturgis, D. Swinehart, and D. Terry, "Epidemic algorithms for replicated database maintenance," in *Proceedings of the Sixth Annual ACM Symposium on Principles of Distributed Computing (PODC)*, 1987, pp. 1–12.
- [20] K. P. Birman and R. van Renesse, "Reliable distributed systems: Technologies, web services, and applications," *Springer*, 2007, chapters on gossip-based failure detection and reliability mechanisms.
- [21] R. van Renesse, K. P. Birman, and W. Vogels, "Astrolabe: A robust and scalable technology for distributed system monitoring, management, and data mining," in *ACM Transactions on Computer Systems*, vol. 21, no. 2, 1998.
- [22] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Voshall, and W. Vogels, "Dynamo: Amazon's highly available key-value store," in *Proceedings of Twenty-first ACM SIGOPS Symposium on Operating Systems Principles (SOSP)*, 2007, pp. 205–220.
- [23] M. Y. Chan and F. Y. L. Chin, "Optimal resilient distributed algorithms for ring election," *IEEE Transactions on Parallel and Distributed Systems*, vol. 4, no. 4, pp. 475–480, 1993.
- [24] M. El-Ruby, J. Kenevan, R. Carlson, and K. Khalil, "A linear algorithm for election in ring configuration networks," in *Proceedings of the 24th Hawaii International Conference on System Sciences (HICSS-24)*, 1991, pp. 117–123.
- [25] D. Conard and A. Ebnenasir, "Prasle: Practical self-stabilizing leader election for iot networks," in *Proceedings of the 7th IEEE European Dependable Computing Conference (EDCC)*. IEEE, 2021, pp. 25–32.
- [26] A. Favier, L. Arantes, J. Lejeune, and P. Sens, "Centrality-based eventual leader election in dynamic networks," in *20th IEEE International Symposium on Network Computing and Applications (NCA)*, 2021.
- [27] M. Burrows, "The chubby lock service for loosely-coupled distributed systems," in *Proc. 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2006.
- [28] L. Lamport, "Paxos made simple," *ACM SIGACT News*, vol. 32, no. 4, pp. 51–58, 2001.
- [29] R. van Renesse and D. Malkhi, "Paxos made moderately complex," *ACM Computing Surveys*, vol. 47, no. 3, 2015.
- [30] M. Isard, "Autopilot: Automatic data center management," *IEEE Computer*, vol. 45, no. 2, pp. 34–40, 2012.
- [31] L. Lamport, D. Malkhi, and L. Zhou, "Cheap paxos," in *Proc. IEEE Int. Conf. Dependable Systems and Networks (DSN)*, 2004.
- [32] B. Oki and B. Liskov, "Viewstamped replication: A new primary copy method to support highly-available distributed systems," in *Proc. 7th ACM Symposium on Principles of Distributed Computing (PODC)*, 1988.
- [33] P. Hunt, M. Konar, F. P. Junqueira, and B. Reed, "Zookeeper: Wait-free coordination for internet-scale systems," in *USENIX Annual Technical Conference (ATC)*, 2010.
- [34] J. Cowling, D. Myers, B. Liskov, R. Rodrigues, and L. Shriram, "Practical paxos in the wild: Experience and lessons," in *Proc. IEEE Symposium on Reliable Distributed Systems (SRDS)*, 2011.
- [35] K. Deshwal, "Raft consensus algorithm: Simplicity and robustness in distributed systems," *European Journal of Computer Science and Information Technology*, vol. 13, no. 14, pp. 184–197, 2025.
- [36] Z. Du, Z. Qu, Y. Fu, M. Huang, and L. Liu, "Multi-strategy-based leader election mechanism for the raft algorithm," *Concurrency and Computation: Practice and Experience*, vol. 35, no. 22, p. e7734, 2023.
- [37] S. Kiefer, A. S. Murawski, J. Ouaknine, and J. Worrell, "On the expected stabilization time of herman's self-stabilizing algorithm," *Theoretical Computer Science*, vol. 412, no. 40, pp. 5658–5667, 2011.
- [38] T. Flatebø and A. Datta, "Self-stabilizing token ring algorithms: Deterministic and probabilistic behaviors," *IEEE Transactions on Software Engineering*, vol. 20, no. 2, pp. 129–137, 1994.
- [39] A. Derhab, N. Badache, and H. Seba, "A self-stabilizing leader election algorithm for fault-prone wireless networks," *IEEE Transactions on Dependable and Secure Computing*, vol. 5, no. 5, pp. 257–271, 2008.
- [40] A. Kandhoul, A. Benslimane, and J. Ben-Othman, "Randomized leader election in dynamic graphs: A self-stabilizing approach," *IEEE Access*, vol. 8, pp. 212 859–212 872, 2020.